

Iteration Guide — Site Inspection App

How to update, extend and redeploy the app

App v1.4 · Form 4.2 · Project 2.1 — Site Assessment Live app: <https://ds-js1.github.io/dryspace-inspect/>

What changed in v1.3. The app is now in a **git repository** with an **automated test suite**, so "save as a new version folder" and the manual test checklist are both retired. Governance moved out of this document into three that are loaded automatically — see §0.

What changed in v1.4. Photo bytes moved into their own IndexedDB store and the database went to version 2 (§3). The app grew its own dialog box (§9). The footer now reports the **build**, not just the version (§4).

0. Read these first

This guide covers **how to change the code**. It deliberately no longer covers what *else* must be updated when you do — that drifted, and now lives elsewhere:

Document	Covers
CLAUDE.md	The non-negotiables. Loaded automatically at the start of every session
05_Release Protocol.md	What to update and when, tiered by change size
docs/DECISION-LOG.md	What was decided and why — read before proposing anything architectural

The decision log matters most. Many questions that look open are already settled, and §3 of it lists decisions that were *reversed* — reverting to any of those reintroduces a known bug.

1. Where everything lives

Source of truth is the repository working folder:

```
01_Contact to Contract\Dryspace Inspection App v1.4\
  index.html          the app – form content, record layer, storage, UI,
                      handover, work-in-progress backup
  ds-media-sync.js     renditions, naming, upload queue, state machine,
                      interrupted-upload recovery, readability pre-flight
  ds-sharepoint.js     Microsoft Graph transport – upload/verify, and from
                      v1.4 list/download/move/putSmall/setFields, all with
                      deadlines
  ds-auth.js           OAuth 2.0 PKCE sign-in, with its own request deadline
  sw.js               offline caching + update detection
  tests.html          550 assertions – run before and after any change
  diagnostics.html    run on a failing device; names the failing hypothesis
  field_ids_v1.4.0.json the field id manifest, named for APP_VER
  manifest.webmanifest, icons
  docs/               decision log, plans, handoffs
  Guides/             generated PDFs – never edited directly
  Training/           staff-facing cards, chart and module
  tools/make_guides.py regenerates Guides/ from the markdown
```

Git internals live **outside** this folder at `C:\Users\jamie\dev\.git-dryspace-inspect`, because the folder sits in a OneDrive-synced SharePoint library and syncing `.git` corrupts it. The `.git` entry here is a pointer file — do

not delete it.

Deployment: GitHub account **DS-JS1**, repository **dryspace-inspect**, live via GitHub Pages from `main`.

Folder naming changed in v1.3. The folder matches the **minor** version (v1.4 today), not every patch. Git holds each version and tags each release, so copying the whole folder for a typo fix creates snapshots nobody reads.

Two files are not in this repository. `Field App Architecture Template.md` and `Dryspace Context Brief.md` moved to `00_AI Tools in Development\_Shared\` on 18 August 2026, which is its own repository, because they are for building sibling apps rather than this one. See the decision log §4.

2. Starting an update session

Point the session at the v1.4 folder so `CLAUDE.md` loads automatically. If that is awkward, start with:

```
Read CLAUDE.md and docs/DECISION-LOG.md in <path>, then ...
```

State the change in your usual list format. Reference fields by their on-form reference number (e.g. 03.4) and their label.

Example prompt

```
In index.html:
(1) Section 05 – add checkbox "Perched water table confirmed by scan" to
    Contributing structural / design factors;
(2) Section 01 – add a B2C/B2B client type dropdown after Client name(s).

Assign data-fid values to the new controls, run tests.html before and after,
update the changelog, bump the versions, and commit.
```

3. The structural rules

Every control needs a permanent `data-fid`

```
<input type="text" data-fid="s3.internal-ceiling-height">
```

That id is what answers are stored against. Set once, when the field is created, and never changed.

- **Renaming a label is safe.** The id does not change.
- **Moving a field is safe.** Same reason.
- **Adding fields, options or sections is safe.**
- **Deleting a field still loses its data** — expected. If it is being *replaced*,

add a migration rule instead.

A control without a `data-fid` is silently excluded from saved data. No error, no warning — the answer simply never saves. `tests.html` fails if any control is missing one.

Every file input needs a permanent `data-mfid`

```
<input type="file" data-mfid="m.s4.wall-moisture-signs" id="wall-signs-media">
```

Added in v1.3. Photos were previously keyed by the input's DOM id, so renaming an id orphaned every photo attached to it — the one place the v1.2 fix had been skipped.

Naming convention

Control type	Pattern	Example
Text, select, textarea	<code>sNN.slug-of-label</code>	<code>s3.internal-ceiling-height</code>
Checkbox	<code>sNN.slug::slug-of-option</code>	<code>s3.slab-design::raft-slab</code>
Radio group	<code>r.groupname</code>	<code>r.grade</code>
"Other" free text	<code>sNN.slug-of-label-other</code>	<code>s3.retaining-wall-other</code>
File input	<code>m.sNN.purpose</code>	<code>m.s4.wall-moisture-signs</code>

Lowercase, hyphenated, derived from the label as it reads when the field is created.

Migration rules — when a field is replaced

Four mechanisms exist. Use them rather than accepting data loss:

1. `data-legacy` — put the old key on the new control to redirect its data.

Multiple rules can be given, separated by `;;`.

2. `VALUE_RULES` — declared as `data-legacy="oldKey=oldValue"`. Ticks a checkbox when an old dropdown held a particular value.

3. `MERGE_RULES` — appends old free text into another field rather than overwriting, and can tick a companion checkbox.

4. `VALUE_REMAP` — rewrites a stored answer when the *wording of an option* changes.

Anything unmatched is preserved under an `unmapped:` prefix rather than discarded.

The trap. Field ids protect against renaming a **label**. They do **not** protect against rewording an **option** in a radio or checkbox group — those are stored as their text. Reword one without a `VALUE_REMAP` and the answer is orphaned on the next autosave.

The record schema — currently 4

The record schema number tracks the migration chain: 1 is label-keyed, 2 introduced field ids, 3 applied the BS8102:2022 grade remap, 4 added the baton pointer for handover through SharePoint. Bump it whenever a migration step is added.

`ensureSchema()` **owns that number and nothing else may set it**. `saveNow()` once pinned `cur.schema` to a literal, which was harmless until the schema moved — after which every record migrated correctly on load and was written straight back at the old number, silently, on every save. Found by accident in v1.4. `saveNow()` now only *preserves* it (`index.html:2322`). If you add a migration step, grep for the number you are leaving behind before you trust it.

IndexedDB — `DB_VERSION` is 2, and there are three stores

`DB_NAME = 'ds-inspections', DB_VERSION = 2` (`index.html:1585`).

Store	Key	Holds
<code>inspections</code>	<code>id</code>	The record: field data, stage, audit trail, baton pointer

Store	Key	Holds
media	mid	One row per photo, video or document — name, type, thumbnail, upload state, the SharePoint identity. Metadata only
bytes	mid	The bytes themselves, as <code>ArrayBuffers</code> : original and report

Version 2 added the bytes store, and `migrateMediaBytes()` (`index.html:4524`) moves any pre-v1.4 record's payload into it at start-up.

Never put a Blob in IndexedDB. Store bytes as an `ArrayBuffer`. A Blob in IndexedDB is a *reference* to a separate file the browser writes and can lose: the record survives, reports the right size, and reading it throws *"The object can not be found here"*. An `ArrayBuffer` is serialised inside the record, so there is nothing to lose track of. Rebuild a Blob only at the moment of use, through `bytesFor()` (`index.html:1671`), and never store it. This cost a real inspection's evidence photo. **D45 was the same fault diagnosed one layer too high; D49 is the settled answer — do not reopen either.**

Two consequences worth knowing before you touch this area:

- **Bytes are separate because `dbAll('media')` runs on every render.**

Deserialising every photo to draw a status panel would be absurd. Metadata is small and read constantly; bytes are large and read twice.

- **A photo whose bytes were already lost is marked as needing retaking, once,**

at migration — rather than surfacing as a mystery at every upload attempt for ever. It is set non-retryable, so it shows as *needs attention* and stays there.

- **Deleting a media record must delete its bytes too** — `forgetMedia()` does

both. Deletion is the one operation that has to know about both stores; missing that left 139 MB stranded on a device with no inspections left at all.

4. Versioning — required on every change

- `index.html`: update `APP_VER`, `VER_DATE`, and `FORM_VER` if form content changed (`index.html:1502–1504`).

- `sw.js`: bump `CACHE_VERSION`. ****This is what makes installed devices pick up the update.**** `tests.html` fails if `CACHE_VERSION` does not contain `APP_VER`.

- `field_ids_v<APP_VER>.json`: the manifest is named for the version, and `tests.html` fails if there is no manifest for the current `APP_VER` or if the `appVersion` / `formVersion` inside it disagree. It is maintained by hand.

- `Guides/*.pdf`: each is named for the version and `tests.html` checks it exists. Regenerate with `python tools\make_guides.py`, which reads `APP_VER` itself.

- `CHANGELOG.txt`: write the entry as you make the change, not afterwards. `tests.html` fails if the changelog does not mention the current `APP_VER`.

- If field ids changed or were added, say so explicitly — Project 3.0 maps against them.
- If any radio or checkbox option was reworded, add a `VALUE_REMAP` and bump the schema.
- Commit, and tag the release (`v1.4.0`).

The home-screen footer and form header read their version from these constants — no separate text edit is needed.

Where the build number comes from — and why it is not APP_VER

The footer line appends `· build 1.4.0-25`, and that string is read from the **live service-worker cache name**, not from a constant (`index.html:4922-4936`).

APP_VER cannot answer "which build is this". A patch folded into an unreleased version leaves APP_VER alone and moves only CACHE_VERSION, so every build since -2 reports v1.4.0 and one screenshot cannot be told from another. Reading the cache name reports what is *actually being served* rather than what somebody remembered to bump.

Ask for the footer build line in any bug report. The version number alone is not enough to know which code someone is running.

5. Testing

Automated. Serve the folder over HTTP and open `tests.html`. **550 assertions** covering field-id integrity, the full v1.1.1 migration chain, the grade remap, the schema upgrade chain, media identity, update detection, handover through SharePoint, naming, the upload queue, the Graph transport, sign-in, and the release gates in §4. **Expect zero failures before you start and zero when you finish.**

Serving matters — `file://` blocks iframe access and IndexedDB. There is a server config at `.claude\launch.json` named `dryspace-inspect`, on port 8765.

The assertion count moves as tests are added. Take the number the suite prints today as the baseline for your session; what matters is that nothing fails and that the count does not *drop*.

Two things the suite says that are easy to misread:

- **"App loaded but its functions are not reachable"** means `index.html` has a syntax error. That is the suite telling you the app is broken, not the suite being broken. It has caught real breakages.
- `instanceof` **across the iframe boundary is always false**. Use `W.Blob`.

Still needs a person. The tests cannot tell you whether the app *feels* right:

1. Create a test inspection and enter data in every changed field.
2. Close the tab, reopen, confirm it reloads intact — including dynamic reveals, Other boxes and the measurement schedule.
3. Export a draft, re-import it, confirm the changed fields survive.
4. Open a record created on the previous version and confirm the migration message appears and old answers are present.
5. Change the stage; confirm the audit trail grows and the exported filename picks it up.
6. Attach a photo; confirm the chip turns green and the count increments.
7. Mark complete, download the report, check the changed fields and measurement totals appear.
8. Delete the test inspection.

Anything touching handover, upload or the baton needs a run against **real SharePoint** as well — the suite exercises the logic, not Graph.

Development caching will bite you. The service worker serves cache-first, so an edit can appear to do nothing. Load `index.html?nosw=1` for a guaranteed fresh copy. This has twice caused the test suite to grade stale code.

6. Redeploying

The repository **is** the deployment. Pushing to `main` publishes the app.

1. Confirm tests pass and version stamps agree.
2. Commit and tag.
3. Merge the working branch (`v1.4`) into `main` and push.
4. **Verify the served file actually changed** — fetch the live `index.html` and confirm `APP_VER` moved. Not that the push succeeded; that the app did.
5. From `v1.3` devices announce the update themselves. Before that, staff had to fully close and reopen the app.

****Pushing to `main` is the release.**** There is no separate deploy step and no way to stage it. Do not push without being asked.

If devices do not update, it is almost always a missed `CACHE_VERSION` bump. Check that first. The tests now catch it, which is why the check exists.

v1.2.0 and v1.2.1 were completed and never deployed, and nobody noticed for two versions because nothing checked that live matched intent. Step 4 is that check.

7. Downstream connection (Project 3.0)

Every report contains a hidden machine-readable JSON block with all answers, section labels, the measurement schedule with totals, per-section completion counts, and the audit trail.

Action required in Project 3.0. From `v1.2` the `rawData` block is keyed by permanent **field id** (e.g. `s3.slab-design`) rather than label text. The Proposal Builder must map against these ids.

`_Shared/Field App Architecture Template.md` — two levels up, in `00_AI Tools in Development/`, and its own repository — describes the whole architecture, written so a sibling app (progress inspections, completion reports, competency assessments, equipment damage) can be built from it. Roughly 80% of the code moves across unchanged — the form is the only part that genuinely differs.

8. Two visual languages, on purpose

The app has **its own dialog box** and it also still uses the **native** ones. That is deliberate and unfinished, not an oversight — do not describe the app as having one dialog style, and do not assume a native `confirm()` you find is a mistake waiting to be tidied.

- `pickFromList(opts)` (`index.html:3528`) — a filterable, tappable list.

Resolves to the chosen item's value, or `null` on Cancel, Escape or a tap on the backdrop. The filter box hides itself below six items, because a search box over two buttons looks like something is missing. Used by *Take over*, *Recover work in progress*, *Browse all inspections*, and the report's fetch-photos-first question.

- `showDialog(opts)` (`index.html:3518`) — the same box used to say

something rather than choose from a list. Takes `lines` (a small definition list of facts) and `choices`. It exists because the handover messages grew to a dozen lines and a native dialog renders those as an undifferentiated wall at whatever size iOS picks. Used by the fork warning and the browse actions.

Still native, roughly in the order they would gain from conversion: the filing nudge before the first upload, the filing gate at handover, the fork warning on import, `warnIfHandedOver()`, and the confirm on opening a handed-over record.

Do not convert anything on a capture-adjacent path without testing on a phone. A custom modal that fails to close is worse than an ugly native one that cannot.

9. Ideas parked for future versions

- **SharePoint List + Power Apps** — the agreed destination for the handover problem. One record per inspection, per-field versioning, no baton discipline needed. Trigger is headcount and volume, not time.
- **B2C/B2B client type indicator** — required by 4.0 for quote type selection.
- **Required-field prompts** — better as stage-transition gates than a blocking check on *Mark complete*.
- **Update the Dryspace Guide to BS8102:2022** — the form moved in v1.2.1 and the Guide now disagrees with it.
- **SOP alignment** — Section 01 corresponds to the New Enquiry SOP, Section 02 to the Discovery Call SOP.
- **Sample photos on fields** — note that hover does not exist on touch devices.
- **App icons rebuilt from the official logo mark** — the current icons are a generic droplet.
- **Document freshness check** — compare each document's modified date against `index.html` and flag any older than the code it describes. The largest remaining manual gap in the release process.
- **Google Photos secondary backup** — scheduled sync reading from SharePoint, for its map-based search. Needs a custom connector.
- **One dialog style** — see §8. The custom box is styled from one place; the work is the conversion and the phone testing, not the styling.
- ****Generate field_ids_v*.json**** — it is maintained by hand, and `tools/` has no script for it. Every other release gate reads `APP_VER` for itself.

Done since this list was last written

- ~~Direct SharePoint upload via Microsoft Graph~~ — built in v1.3.
- ~~The record itself moving through SharePoint~~ — built in v1.4. The share sheet survives only as the offline fallback.
- ~~Automatic backup of in-progress work~~ — built in v1.4, to `wip/`.
- ~~Photo bytes out of the record and out of Blob storage~~ — built in v1.4, as the `bytes` store (§3).
- **Resume a broken upload from where it stopped.** The transport creates a fresh

upload session on every attempt, so a failed 5 MB photo restarts from byte 0. Persisting the session URL on the record would fix it, but it changes what the record stores, so it belongs with a schema move rather than on its own.

- **Clear out old wip/ files.** Nothing deletes them, by design — the app never deletes from SharePoint. They accumulate one per device per inspection. Harmless but untidy; a retention job or a manual sweep at close-out.